Traffic Volume Forecasting Using Deep Learning: LSTM, GRU, and CNN-LSTM Models

Dedeepya Poreddy
dedeepya.poreddy0698@gmail.com
December 2025

Abstract—Predicting traffic accurately is important for modern transportation systems, as it helps manage congestion and plan better routes. This study compares three deep learning models to find the most effective one for forecasting traffic flow: Long Short Term Memory (LSTM), Gated Recurrent Units (GRU), and a hybrid CNN-LSTM model. By testing these models on the PeMS03 dataset, the analysis examines how well they capture both trends over time and patterns across different locations. The results show that while the Stacked GRU model trains the fastest at 213 seconds, the hybrid CNN-LSTM model provides the most accurate predictions with a Mean Absolute Error of 249.84. This finding confirms that combining convolutional layers to extract important features with recurrent layers to model sequences is a highly effective approach for traffic forecasting [Brownlee, 2018].

I INTRODUCTION

The rapid expansion of urban infrastructure has created a strong need for reliable traffic prediction methods. Traffic forecasting is essential for Intelligent Transportation Systems and supports optimal traffic control, routing, and congestion management [Hochreiter and Schmidhuber, 1997]. Traditional statistical models often struggle to handle the stochastic and nonlinear characteristics of traffic data. However, deep learning, particularly Recurrent Neural Networks, provides a robust alternative due to its capacity to model sequential dependencies over extended temporal horizons.

The PeMS03 dataset utilized in this study comprises 151 traffic sensors spanning 21.8 years of hourly data from 2002 to 2024, collected across California District 03. This dataset presents significant challenges due to nonlinear traffic behavior, abrupt spikes during peak hours, and complex spatial correlations among sensors. The objective is to predict next hour traffic volume using 12 hours of historical data across all sensors, comparing three deep learning architectures.

To address these challenges, this paper evaluates three distinct deep learning frameworks:

1. **Stacked LSTM**: A deep recurrent architecture designed to mitigate the vanishing gradient problem through gating mechanisms and memory cells [Hochreiter and Schmidhuber, 1997, Gers et al., 2000].

2. **Stacked GRU**: A simplified variant of LSTM that reduces computational complexity by merging gating operations while maintaining competitive performance [Cho et al., 2014].

3. **Optimized CNN LSTM**: A hybrid architecture that employs one dimensional convolutional layers for feature extraction, followed by LSTM layers to model temporal patterns [Donahue et al., 2015, Brownlee, 2018].

The study provides a comprehensive examination of these model architectures, including their parametric complexity, memory utilization, and layer configurations, and presents a rigorous performance comparison using Mean Absolute Error and Root Mean Squared Error metrics.

II RELEVANT PAST WORK

Research in traffic forecasting has progressively shifted toward deep learning methodologies because these models demonstrate superior capability in processing large scale, high dimensional, and noisy datasets compared to traditional statistical approaches.

- **LSTM Networks:** Conventional Recurrent Neural Networks frequently encounter difficulties in preserving information over extended sequences due to the vanishing gradient phenomenon. To address this limitation, Hochreiter and Schmidhuber [1997] introduced the Long Short Term Memory architecture, which incorporates specialized gating mechanisms and a dedicated memory cell. These components operate synergistically to regulate information retention, forgetting, and propagation. Consequently, LSTM networks excel at capturing long range dependencies in traffic data, where historical patterns and recurrent behaviors substantially influence future conditions. This foundational work provides the theoretical basis for the Stacked LSTM implementation in the current study. Subsequently, Gers et al. [2000] enhanced the architecture with forget gates, further refining its capacity to selectively retain or discard information, thereby influencing contemporary LSTM implementations in modern research.

- **GRU Networks:** Cho et al. [2014] proposed Gated Recurrent Units as a computationally efficient alternative to LSTM networks. GRUs consolidate specific gates, particularly the input and forget mechanisms, into a unified update gate. This architectural simplification reduces the parameter count and accelerates training while maintaining performance comparable to LSTM networks. GRUs are typically preferred when computational efficiency is prioritized or when working with moderately sized datasets. The GRU architecture establishes the theoretical foundation for the Stacked GRU model implementation in this investigation.

- **Hybrid Models:** Contemporary research indicates that hybrid deep learning architectures, such as CNN LSTM models, can achieve superior forecasting accuracy. Donahue et al. [2015] pioneered CNN LSTM hybrid architectures through long term recurrent convolutional networks for visual recognition and description tasks. These models leverage Convolutional Neural Networks to extract localized and spatially relevant features, including abrupt fluctuations or regional patterns from raw input data. The extracted representations are subsequently processed by LSTM layers, which model the temporal evolution of these patterns. This integrated approach enables the model to capture both spatial and temporal dependencies, rendering it particularly suitable for complex traffic prediction tasks. Brownlee [2018] provided practical implementation guidance for CNN LSTM architectures, informing the model development methodology adopted in the current study.

- **Traffic Prediction Comparative Studies:** Das [2023] conducted a recent comparative analysis of LSTM and GRU networks for traffic volume prediction using memory based recurrent neural networks. This comparative investigation provides important contextual framework for experimental design and methodology, validating the significance of evaluating multiple recurrent architectures for time series forecasting applications.

III APPROACH

A Why This Comparison Matters

This project compares three commonly used deep learning models for traffic volume prediction: LSTM, GRU, and CNN LSTM. Traffic data contains complicated patterns that shift over time, including gradual trends, repeating daily cycles, and sudden irregular spikes. Because these patterns vary in both scale and frequency, it is not immediately obvious which model works best for capturing them.

Traffic flow is highly dynamic with sudden peaks and fluctuations that traditional models cannot capture effectively. The PeMS03 dataset includes 151 sensors, making it important to model both temporal patterns and spatial relationships across stations. LSTM and GRU are strong choices for learning time dependent patterns, but they differ in complexity and efficiency, which makes a direct comparison worthwhile. CNN LSTM adds the ability to learn spatial correlations, potentially improving accuracy during peak congestion.

LSTM and GRU models are widely used for time series prediction because they can learn long term relationships in sequential data. However, they may sometimes overlook short term local patterns that matter in traffic flow, such as brief increases or sudden drops caused by unexpected events.

CNN LSTM models include an extra convolutional stage that can learn to detect these short term variations before passing the processed information to the LSTM part for temporal learning. By including this model alongside LSTM and GRU, the project aims to understand whether the additional feature extraction stage leads to better prediction performance or whether the simpler recurrent models are sufficient on their own.

Overall, the motivation for this comparison is to evaluate how these three models handle the different characteristics of traffic volume data and to determine which architecture performs best for this specific prediction task.

B Data Preparation and Processing

The study uses the PeMS03 traffic dataset, which contains 191,136 hourly records collected from 151 traffic sensors across California District 03. The data covers a period of approximately 21.8 years, from June 2002 to March 2024.

To ensure high data quality, a careful preprocessing pipeline is applied. First, timestamps are removed to focus only on traffic values. Missing entries are then filled in through interpolation to keep the sequences continuous. To reduce noise, the Interquartile Range method is used to detect and remove statistical outliers. Finally, the data undergoes MinMax scaling to adjust all values between zero and one, which helps the models train more stably.

The dataset is split into three distinct sets based on time. The training set includes 152,898 sequences from June 2002 to October 2019, covering about 17.4 years. The validation set contains 19,101 sequences from October 2019 to December 2021. The remaining 19,101 sequences, from December 2021 to March 2024, form the testing set. This time based split ensures that model evaluation is based on predicting future data rather than mixing past and future information.

C Model Design and Structure

Three distinct architectures were implemented using the TensorFlow Keras framework. Each model was designed to process an input sequence of 12 time steps across 151 sensors, predicting traffic volume for the subsequent hour.

C.1 Stacked Long Short Term Memory Network

The Stacked LSTM model employs memory cells and gating mechanisms to regulate information flow, following the foundational architecture described by Hochreiter and Schmidhuber [1997]. The implementation consists of two stacked LSTM layers, where the first layer processes the input sequence and propagates the complete hidden state history to the second layer. This stacked configuration enables deep temporal feature extraction by allowing the model to learn representations at multiple timescales. Dropout layers are inserted after each LSTM layer to mitigate overfitting. This model requires approximately 791 KB of memory and maintains a balanced capacity for learning complex temporal dependencies.

- **Input Shape**: (12, 151)
- **Total Parameters**: 202,583

Table 1: Model Summary: Stacked LSTM

Layer (Type)	**Output Shape**	**Param #**
LSTM (Layer 1)	(None, 12, 128)	143,360
Dropout (0.2)	(None, 12, 128)	0
LSTM (Layer 2)	(None, 64)	49,408
Dropout (0.2)	(None, 64)	0
Dense (Output)	(None, 151)	9,815

C.2 Stacked Gated Recurrent Unit Network

The Stacked GRU model follows a similar hierarchical structure to the LSTM but utilizes Gated Recurrent Units as introduced by Cho et al. [2014]. By consolidating the gating mechanisms into update and reset gates, this architecture reduces the parameter count while maintaining the capacity to capture sequential dependencies. The simplified internal dynamics result in a smaller computational footprint and faster training times compared to LSTM, making this model particularly suitable when computational efficiency is prioritized. It is the most memory efficient among the three architectures, requiring only 605 KB of memory.

- **Input Shape**: (12, 151)
- **Total Parameters**: 154,967

Table 2: Model Summary: Stacked GRU

Layer (Type)	**Output Shape**	**Param #**
GRU (Layer 1)	(None, 12, 128)	107,904
Dropout (0.2)	(None, 12, 128)	0
GRU (Layer 2)	(None, 64)	37,248
Dropout (0.2)	(None, 64)	0
Dense (Output)	(None, 151)	9,815

C.3 Hybrid Convolutional LSTM Network

This hybrid architecture integrates convolutional operations with recurrent sequence modeling, drawing inspiration from Donahue et al. [2015] and practical implementation guidance from Brownlee [2018]. The model begins with a reshaping operation to prepare the data for convolutional processing. A Time Distributed 1D Convolutional layer then scans the input window, applying convolutional filters to extract localized features from each time step independently. These extracted features capture short range patterns and spatial correlations among sensors. The features are subsequently flattened and passed through dropout regularization before being processed by an LSTM layer, which models the temporal evolution of these extracted representations. Due to the high dimensionality introduced by the convolutional feature maps, this model requires substantially more memory, approximately 9.54 MB. The spatial temporal processing capability of this architecture makes it particularly well suited for capturing both inter sensor correlations and sequential dynamics in traffic data.

- **Input Shape**: (12, 151)
- **Total Parameters**: 2,500,567

Table 3: Model Summary: Optimized CNN LSTM

Layer (Type)	**Output Shape**	**Param #**
Reshape	(None, 12, 151, 1)	0
TimeDist (Conv1D)	(None, 12, 151, 64)	128
TimeDist (Flatten)	(None, 12, 9664)	0
Dropout (0.1)	(None, 12, 9664)	0
LSTM	(None, 64)	2,490,624
Dropout (0.1)	(None, 64)	0
Dense (Output)	(None, 151)	9,815

D Model Training and Experimental Setup

A sliding window approach was used to transform the time series forecasting task into a supervised learning problem. For the primary experiment, a historical lookback period of 12 time steps, equivalent to 12 hours of data, was used to predict traffic volume for the subsequent hour. This window length ensures that each model has access to at least half a day of recent traffic dynamics when making each prediction.

The training process was conducted for 15 epochs with a batch size of 64. The Adam optimizer was applied with a learning rate of 0.001. To prevent overfitting, an early stopping mechanism monitored the validation loss with a patience of 3 epochs, restoring the best model weights when training terminated. Mean Squared Error served as the primary loss function for optimization. An identical training configuration was maintained across all three models, the Stacked LSTM, Stacked GRU, and Hybrid CNN LSTM, to ensure a fair and consistent comparison.

To investigate how the amount of historical context influences prediction quality and computational cost, the experiments were repeated with additional lookback windows of 8, 16, and 24 hours while keeping all other hyperparameters fixed. The 8 hour window focuses on short term intraday patterns, capturing dynamics within a typical work shift or morning period. The 16 hour window extends the context beyond a single rush period, encompassing both peak and off peak hours. The 24 hour window captures full daily cycles and repeated congestion patterns from the previous day, allowing models to leverage periodicity in

traffic behavior. This multi window experimental design makes it possible to observe whether longer context consistently improves accuracy or whether diminishing returns appear as the sequence length grows.

Figure 1 provides a visual overview of this complete experimental pipeline, from data preprocessing through model evaluation.

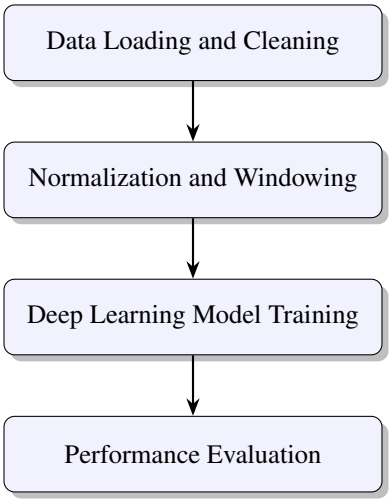

Figure 1: Methodology

IV RESULTS

A Performance Measures

Model performance was evaluated using two standard regression metrics: Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). MAE provides a straightforward measure of average prediction error in the original units of traffic volume, while RMSE places greater emphasis on larger errors, making it particularly useful for assessing model sensitivity to significant deviations or peak period mispredictions.

B Main Findings

Table 4 presents a comparative analysis of the three architectures. The Stacked GRU model demonstrated the fastest training convergence, completing the optimization process in 213.12 seconds. However, the Hybrid CNN LSTM model achieved the lowest error rates across both evaluation metrics, indicating its superior capability to capture the complex spatiotemporal structure inherent in traffic data.

Table 4: Detailed Comparison of Model Architectures and Performance.

Feature	**LSTM**	**GRU**	**CNN LSTM**
Mechanism	Gated Memory Cells	Simplified Gating	Conv. Features + RNN
Params	$\sim$202k	$\sim$155k	$\sim$2.5M
Memory Usage	791 KB	605 KB	9.54 MB
Training Time	255.58s	**213.12s** (Fastest)	595.38s
MAE Score	263.44	264.11	**249.84** (Best)
RMSE Score	519.39	517.60	**491.05** (Best)

Table 5: Model performance across different lookback windows. MAE and RMSE are computed on the test set; training time is measured per model configuration.

Lookback	Model	MAE	RMSE	Training Time (s)
8 hours	LSTM	265.63	518.52	219.58
	GRU	255.25	506.76	256.43
	CNN LSTM	246.53	482.51	424.01
12 hours	LSTM	263.44	519.39	255.58
	GRU	264.11	517.60	213.12
	CNN LSTM	249.84	491.05	595.38
16 hours	LSTM	264.45	528.34	252.89
	GRU	260.33	504.76	183.94
	CNN LSTM	245.88	486.93	935.33
24 hours	LSTM	261.91	513.86	375.11
	GRU	254.65	501.63	403.49
	CNN LSTM	247.53	484.32	1542.36

Influence of Historical Context Length Beyond the standard 12 hour configuration, the models were tested with lookback windows of 8, 16, and 24 hours to examine how the length of historical input affects both prediction accuracy and computational expense. Table 5 presents these results, showing that the Hybrid CNN LSTM consistently achieves the lowest MAE and RMSE across all four window sizes. This consistency confirms the model's robustness to variations in input sequence length. However, training time increases substantially as the lookback window expands, with the most pronounced increase observed for the CNN LSTM at 24 hours. This trend reflects the higher computational cost associated with processing longer input sequences, particularly for architectures that combine convolutional and recurrent operations.

Performance Analysis Across Lookback Windows

The following subsections provide detailed visual and quantitative results for each lookback window configuration, including 8, 12, 16, and 24 hours. This structured analysis demonstrates how the amount of historical context influences both predictive performance and computational efficiency.

Eight Hour Lookback Window

The 8 hour window focuses on short term intraday patterns, capturing approximately one third of a full daily cycle. Notably, this shorter context produced highly competitive results across all models. As shown in Figure 4, the Hybrid CNN LSTM achieved a lower MAE of 246.53 with this configuration than it did with the 12 hour baseline, suggesting that recent traffic dynamics, those within a typical work shift or morning period, carry strong predictive signal. Figure 2 provides visual confirmation of this finding, demonstrating that the model successfully captures immediate traffic peaks and fluctuations using only recent history without requiring a full day of context.

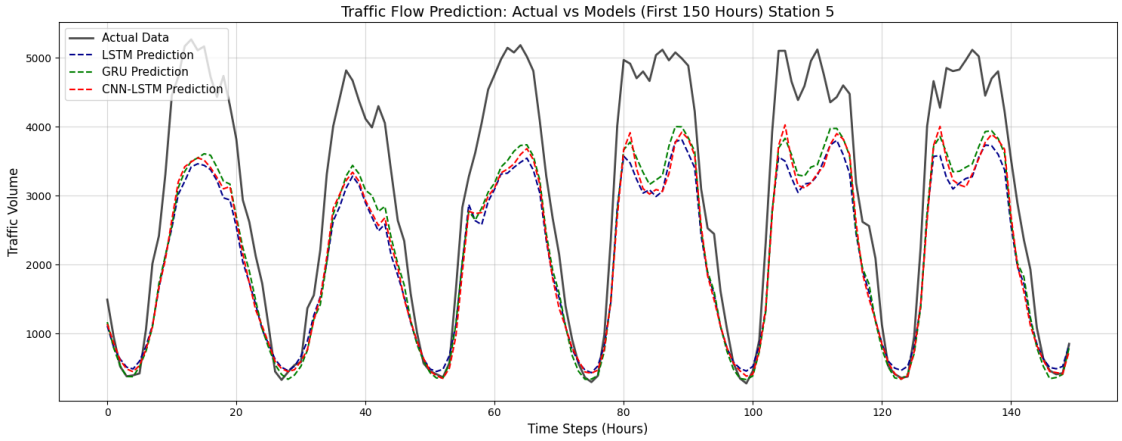

Figure 2: Traffic Flow Prediction for Station 5 (8-Hour Lookback). The model captures immediate trends effectively.

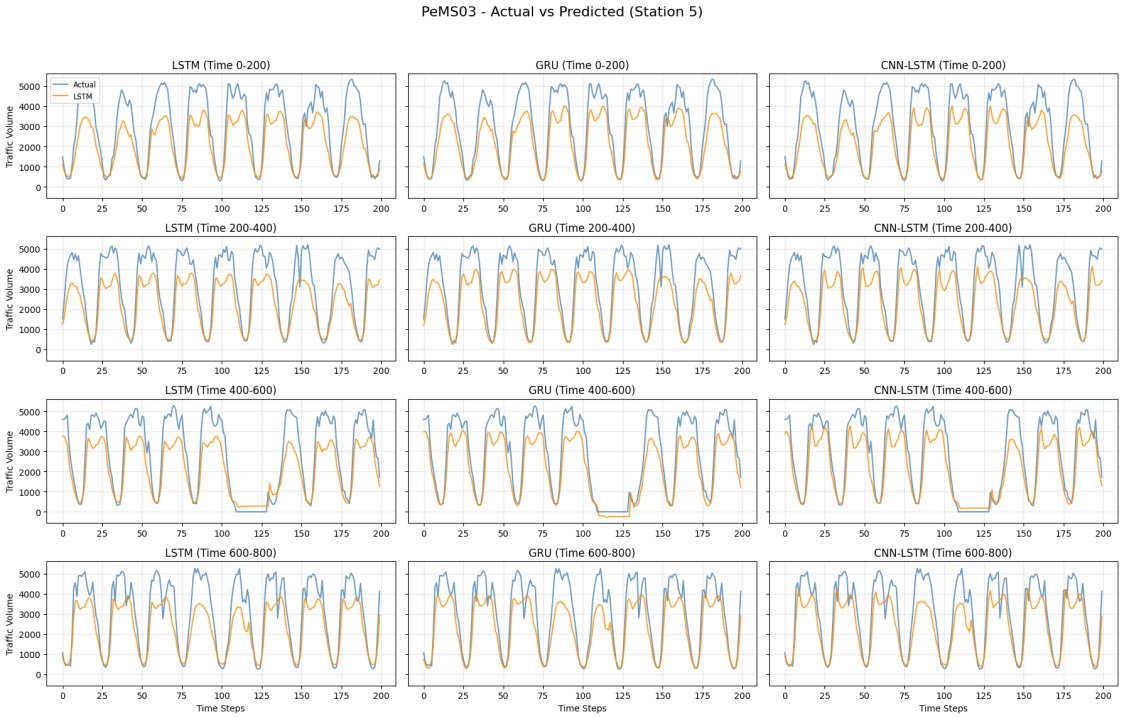

Figure 3: Detailed actual vs. predicted comparison for Station 5 (8-Hour Lookback).

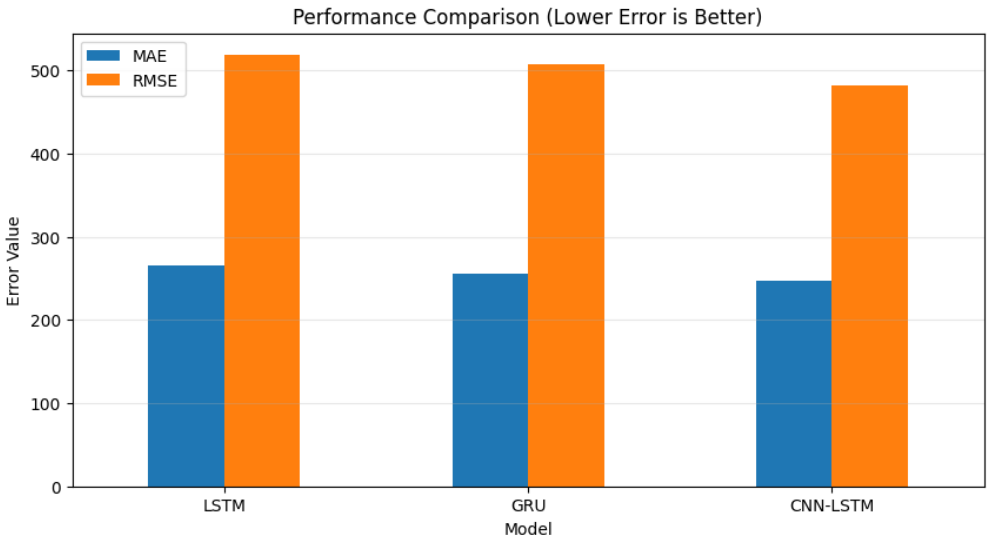

Figure 4: Performance Comparison for 8-Hour Lookback: CNN LSTM performs remarkably well, outperforming the 12-hour baseline.

Twelve Hour Lookback Window

Figure 5 illustrates the predictive performance for the baseline 12 hour window configuration. The Hybrid CNN LSTM, represented by the red dashed line, continues to show the closest alignment with actual traffic peaks compared to the other architectures. However, the error metrics for this configuration are slightly higher than those observed for both the 8 hour and 16 hour windows. This finding suggests that for the PeMS03 dataset, a 12 hour context may introduce transitional noise, periods that contain less predictive signal, which marginally reduces the hybrid model's precision relative to other window sizes.

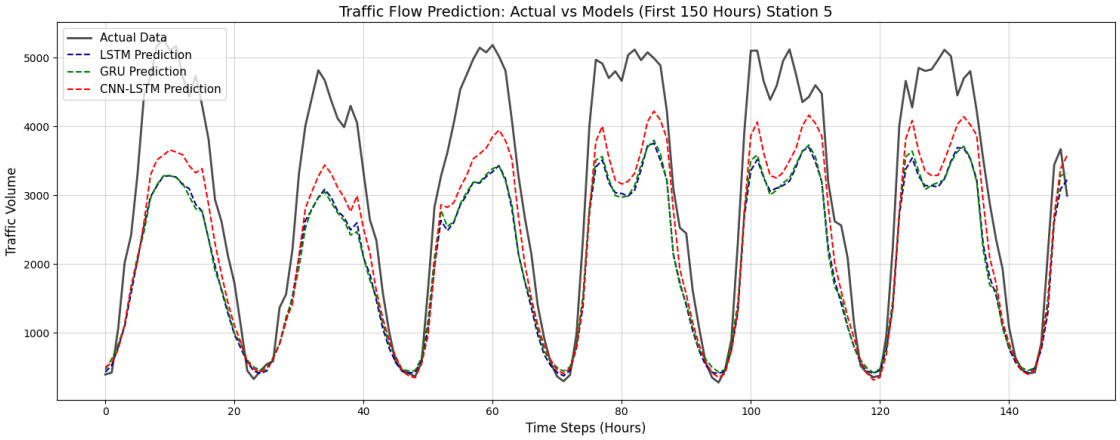

Figure 5: Traffic Flow Prediction for Station 5 (12-Hour Lookback).

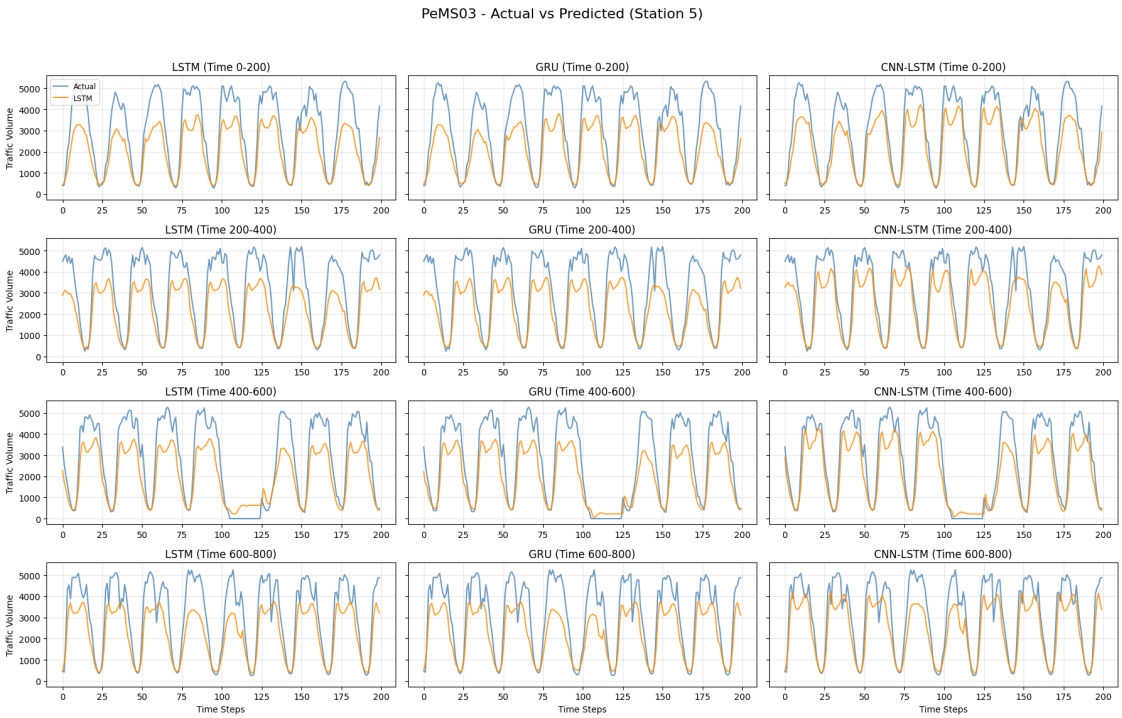

Figure 6: Detailed actual vs. predicted comparison for Station 5 (12-Hour Lookback).

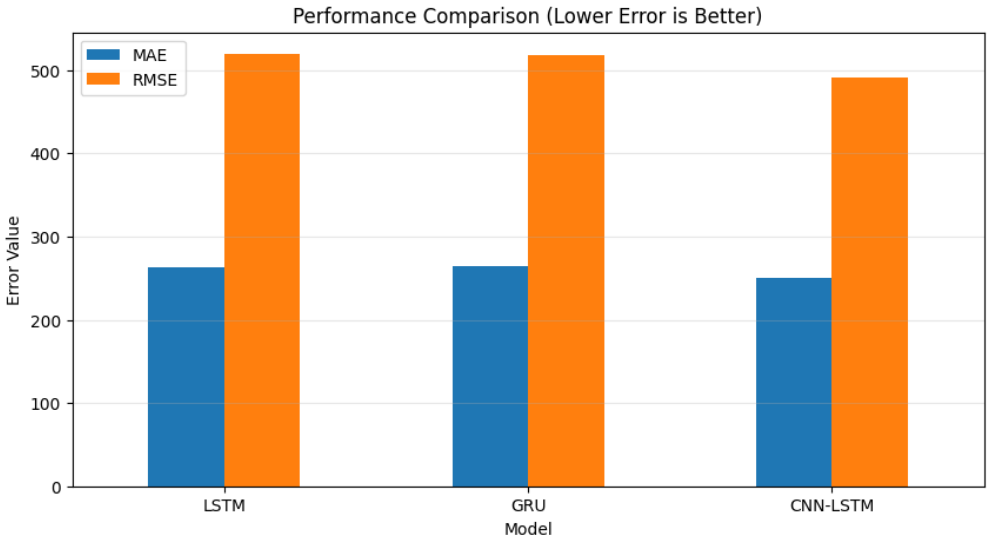

Figure 7: Performance Comparison for 12-Hour Lookback: The CNN LSTM remains the most accurate model.

Sixteen Hour Lookback Window

The 16 hour window emerged as the optimal configuration for the Hybrid CNN LSTM model. As shown in figure 10, this architecture achieved its lowest overall Mean Absolute Error of 245.88 and correspondingly improved Root Mean Squared Error with this extended context. Extending the historical lookback beyond the immediate 12 hour period appears to provide valuable trend information spanning multiple peak periods, enabling the model to better anticipate complex congestion patterns. Figure 8 confirms this finding visually, demonstrating robust alignment between predicted and actual values during both peak and off peak hours.

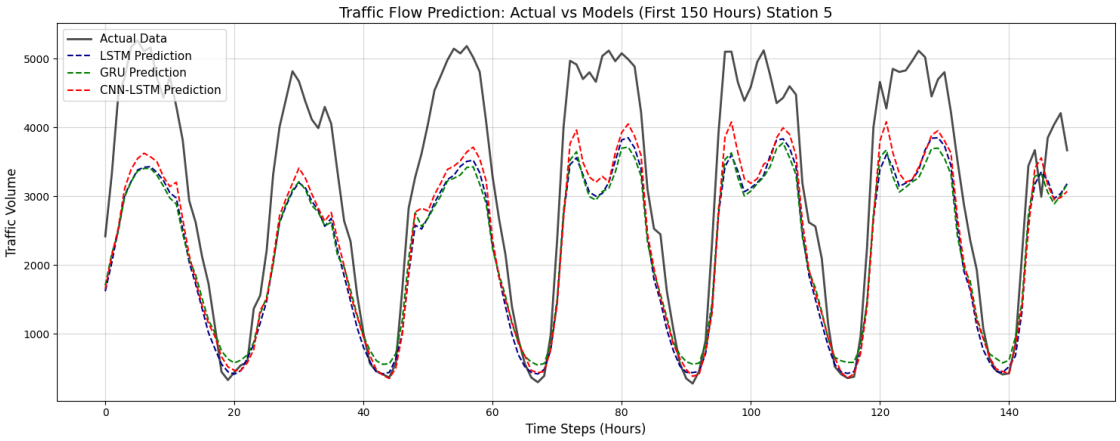

Figure 8: Traffic Flow Prediction for Station 5 (16-Hour Lookback). This window yields the best alignment for CNN LSTM.

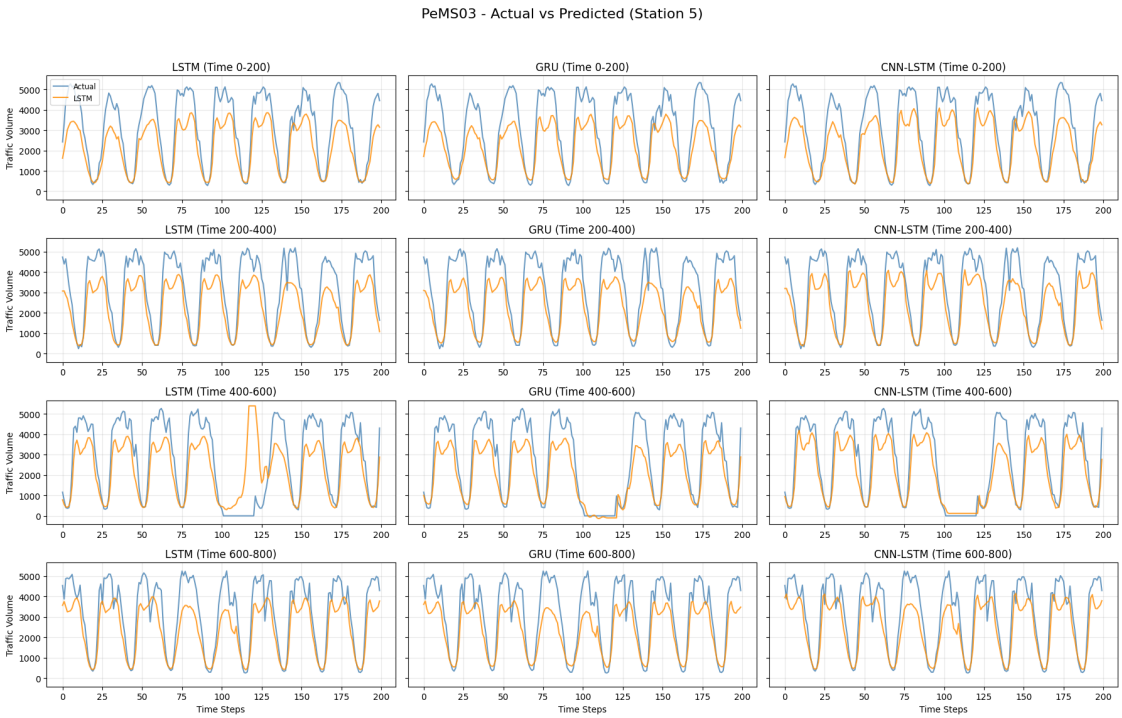

Figure 9: Detailed actual vs. predicted comparison for Station 5 (16-Hour Lookback).

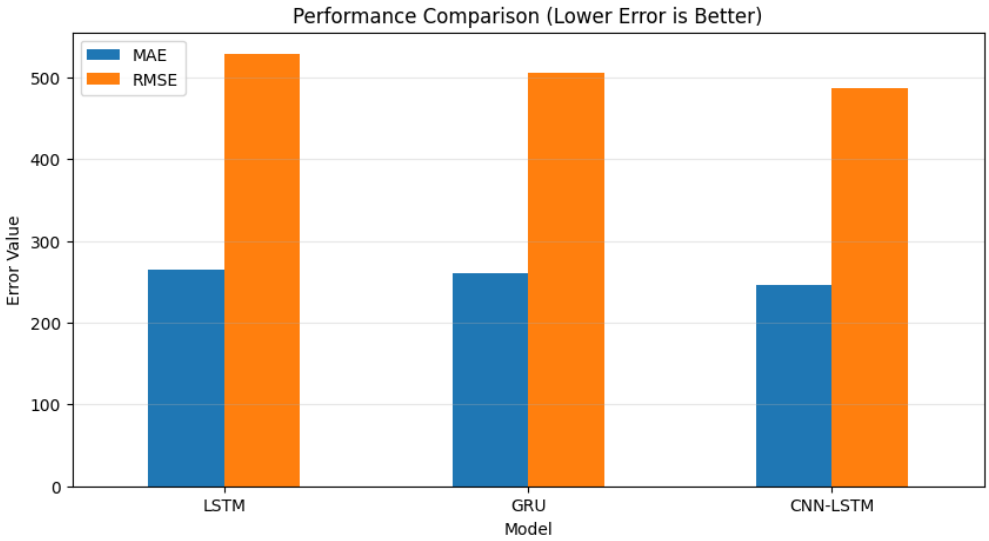

Figure 10: Performance Comparison for 16-Hour Lookback: Optimal performance for the CNN LSTM architecture.

Twenty Four Hour Lookback Window

The 24 hour window captures a complete daily cycle, including morning peak, evening peak, and overnight off peak periods. Interestingly, this extended context benefits the pure recurrent models most substantially. Both the Stacked LSTM and Stacked GRU achieved their lowest respective MAE scores with this configuration, recording 261.91 and 254.65. This improvement suggests that having access to a full day of historical patterns enables these architectures to better leverage daily periodicity. However, for the Hybrid CNN LSTM, prediction accuracy slightly degraded compared to the 16 hour window, and the computational cost became prohibitive. Training time spiked to over 1500 seconds, more than three times longer than the 16 hour configuration, reflecting the compounding complexity introduced by processing longer sequences through both convolutional and recurrent layers, as illustrated in Figure 13.

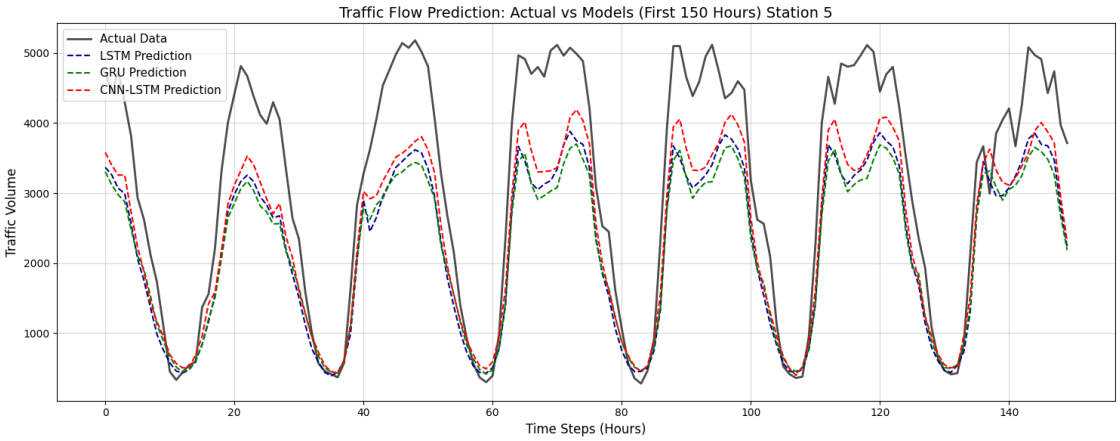

Figure 11: Traffic Flow Prediction for Station 5 (24-Hour Lookback).

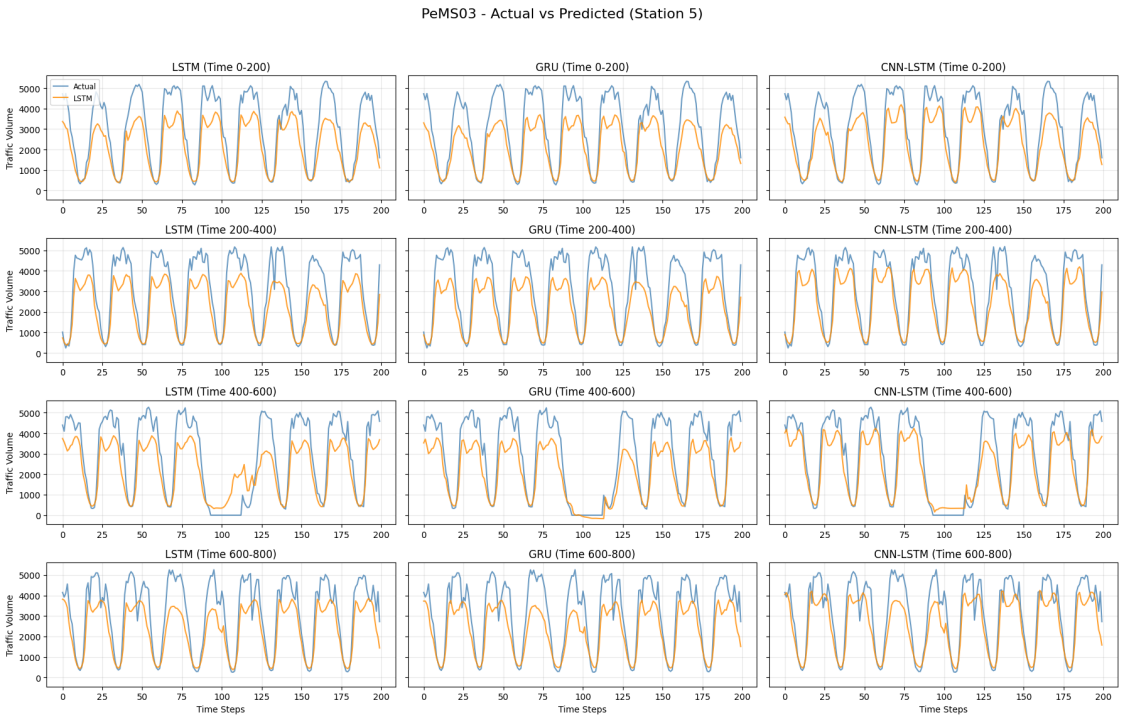

Figure 12: Detailed actual vs. predicted comparison for Station 5 (24-Hour Lookback).

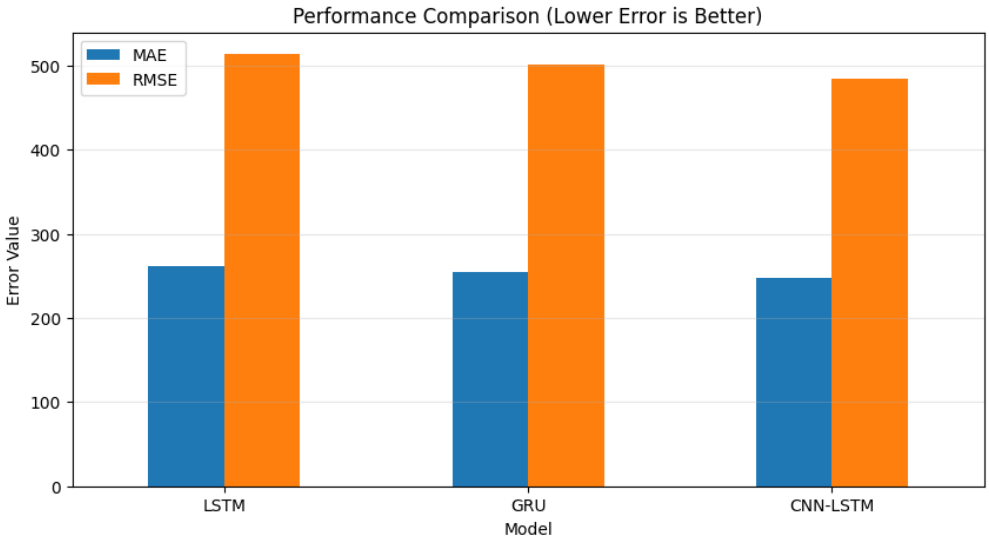

Figure 13: Performance Comparison for 24-Hour Lookback: Best setting for LSTM/GRU, but computationally expensive for CNN LSTM.

C Constraints and Trade Offs

Although the Hybrid CNN LSTM delivers the best prediction accuracy, it presents practical challenges regarding computational resources. The model contains approximately 2.5 million trainable parameters, making it substantially larger than the Stacked LSTM, which has roughly 202,000 parameters, and the Stacked GRU, which has approximately 155,000 parameters.

This architectural complexity creates a clear trade off between accuracy and efficiency. While the Hybrid CNN LSTM achieves the lowest MAE and RMSE across most configurations, it is computationally expensive to train and deploy. In our experiments, training the Hybrid CNN LSTM required between 424 seconds for the 8 hour window and over 1540 seconds for the 24 hour window. By contrast, the Stacked GRU model remained highly efficient throughout, training in as little as 183 seconds while maintaining competitive prediction performance.

Due to these computational demands, the Hybrid CNN LSTM is best suited for server side or cloud based systems with access to robust computing infrastructure. Conversely, the Stacked GRU offers an excellent balance of speed and accuracy, making it the ideal candidate for resource constrained environments such as mobile devices or edge computing platforms. The Stacked LSTM occupies a middle ground, less parameter heavy than the hybrid model but slower than the GRU, making it a reasonable choice when moderate computational resources are available and long term dependency learning remains a priority.

Overall, while the Hybrid CNN LSTM is the superior choice when raw accuracy is the primary objective, the Stacked GRU remains a strong contender for real time applications where inference speed, training time, and model size are critical operational constraints.

V Conclusion

This study presented a comparative analysis of Stacked LSTM, Stacked GRU, and Hybrid CNN LSTM architectures for traffic volume prediction using the PeMS03 dataset. The results demonstrate that while standard recurrent networks provide reliable baselines, the Hybrid CNN LSTM model delivers the highest accuracy by effectively combining spatial feature extraction through convolutional operations with temporal sequence modeling through recurrent layers. Our experiments revealed that the Hybrid CNN LSTM performs optimally with a 16 hour lookback window, successfully capturing complex traffic patterns that pure recurrent architectures tend to smooth over or misinterpret. However, this performance advantage comes with substantial computational cost: the hybrid model requires significantly more training time, up to four times longer than the Stacked GRU for larger input windows. For applications where computational resources are sufficient and prediction precision is paramount, the Hybrid CNN LSTM is the preferred architecture. For embedded systems, real time applications, or scenarios requiring rapid model deployment, the Stacked GRU model offers the most efficient alternative, providing near LSTM accuracy with a fraction of the parameter footprint. Future research should focus on optimizing the Hybrid CNN LSTM architecture, perhaps through dimensionality reduction techniques or more efficient convolutional designs, to retain its superior accuracy while minimizing computational overhead.

References

Jason Brownlee. Cnn long short-term memory networks, 2018. URL https://machinelearningmastery.com/cnn-long-short-term-memory-networks/. Machine Learning Mastery.

Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.

L. C. Das. Traffic volume prediction using memory-based recurrent neural networks: A comparative analysis of lstm and gru. *arXiv preprint arXiv:2303.12643*, 2023. URL https://arxiv.org/abs/2303.12643.

Jeffrey Donahue, Lisa Anne Hendricks, Sergio Guadarrama, Marcus Rohrbach, Subhashini Venugopalan, Kate Saenko, and Trevor Darrell. Long-term recurrent convolutional networks for visual recognition and description. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2625–2634, 2015.

Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins. Learning to forget: Continual prediction with lstm. *Neural Computation*, 12(10):2451–2471, 2000.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.